

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

3. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: 28×28

4. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Perform the following preprocessing:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to $[0,1]$.
5. Convert labels into one-hot vectors.

5. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Task 5: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

6. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks

1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

7. Source Code

The complete source code for this experiment is available on GitHub:

https://github.com/srinithi0406/Deep_Learning/tree/main/DL_2

8. Mandatory Plots

Generate the following plots.

1. Sample Images
 2. Class Distribution
 3. Training Accuracy vs Epoch
 4. Validation Accuracy vs Epoch
 5. Training Loss vs Epoch
 6. Validation Loss vs Epoch
 7. Confusion Matrix
 8. Hyperparameter Search Results
 9. Best Model Accuracy Comparison
- Write a 2–3 line inference for each plot.

1. Sample Images

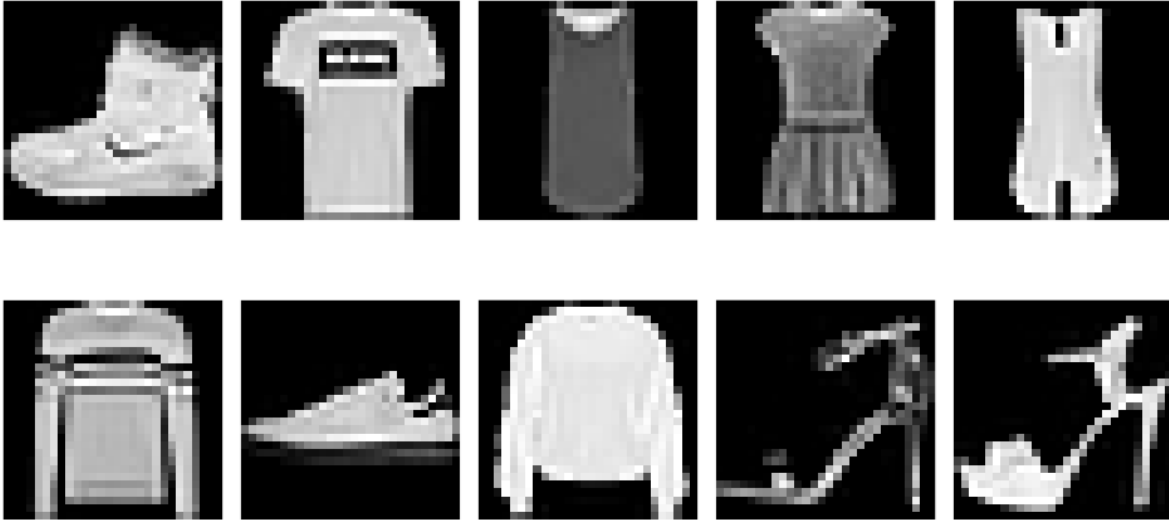


Figure 1: Sample Images from the Fashion-MNIST Dataset

Inference: The dataset is containing different types of clothes like T-shirts, dresses, shoes and bags. All images are in black and white and have the same size of 28×28 pixels. From these sample images I understood what the model has to classify.

2. Class Distribution

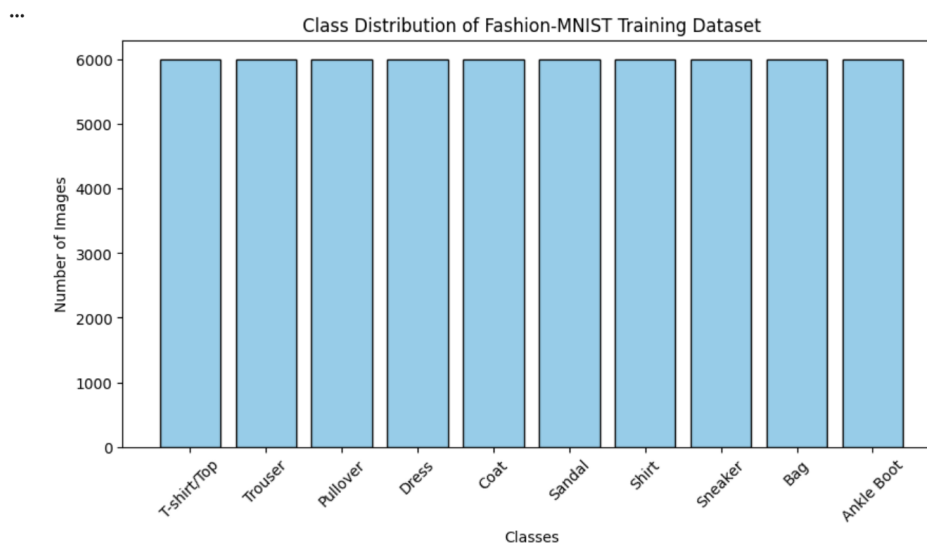


Figure 2: Class Distribution of the Fashion-MNIST Training Dataset

Inference: All classes are having almost 6000 images each. As every class has same amount of data, the model will get equal examples to learn every clothing category.

3. Training and Validation Accuracy vs Epoch

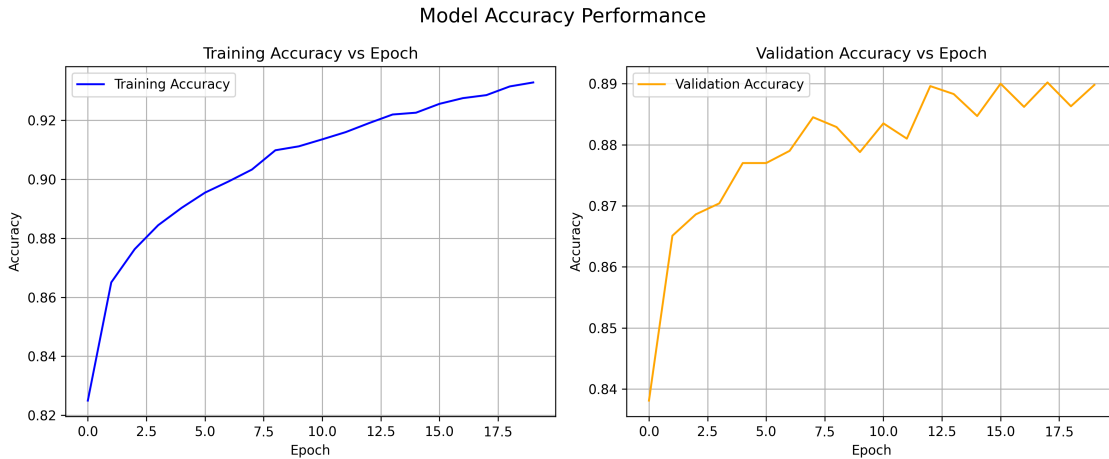


Figure 3: Training and Validation Accuracy vs Epoch

Inference: Training Accuracy: As the epochs increased, the training accuracy has also increased from about 82% to around 93%. This shows that the model has learned better from the training data after every epoch. **Validation Accuracy:** The validation accuracy also improved well and reached nearly 89%. After few epochs it became almost stable which shows that the model is also giving good results on unseen test data.

4. Training and Validation Loss vs Epoch

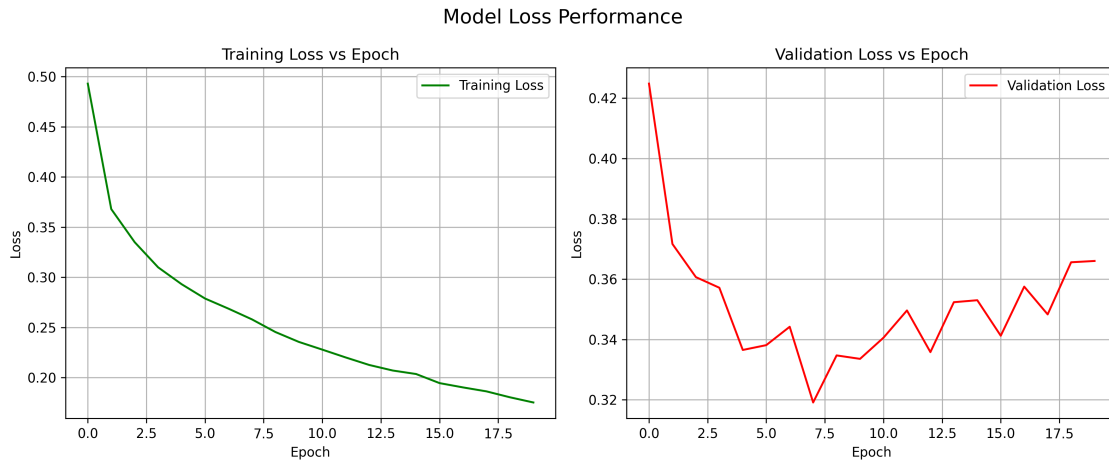


Figure 4: Training and Validation Loss vs Epoch

Inference: Training Loss: The training loss reduced continuously from about 0.49 to around 0.18. This indicates that the model made only fewer mistakes while learning from the training data. **Validation Loss:** The validation loss decreased at the beginning and then changed only slightly in the later epochs. Even though there are small fluctuations, the overall performance remains good.

5. Confusion Matrix

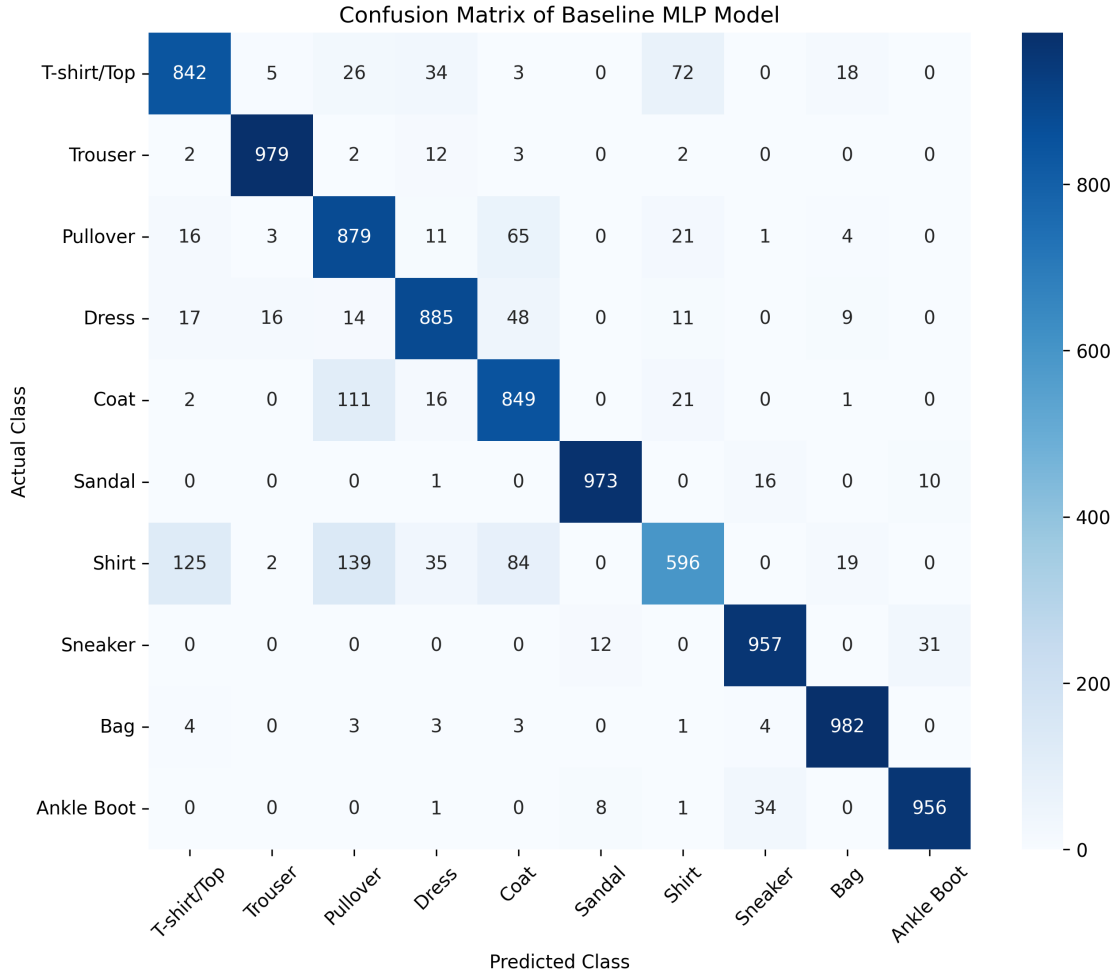


Figure 5: Confusion Matrix of the Baseline MLP Model

Inference: Most of the correct predictions are present along the diagonal. This shows that the model has classified many images correctly. I also noticed that the model confused some similar clothes like Shirts, T-shirts, Pullovers and Coats more often. Classes like Trouser, Sandal, Bag and Ankle Boot were classified correctly in most of the cases.

6. Hyperparameter Search Results

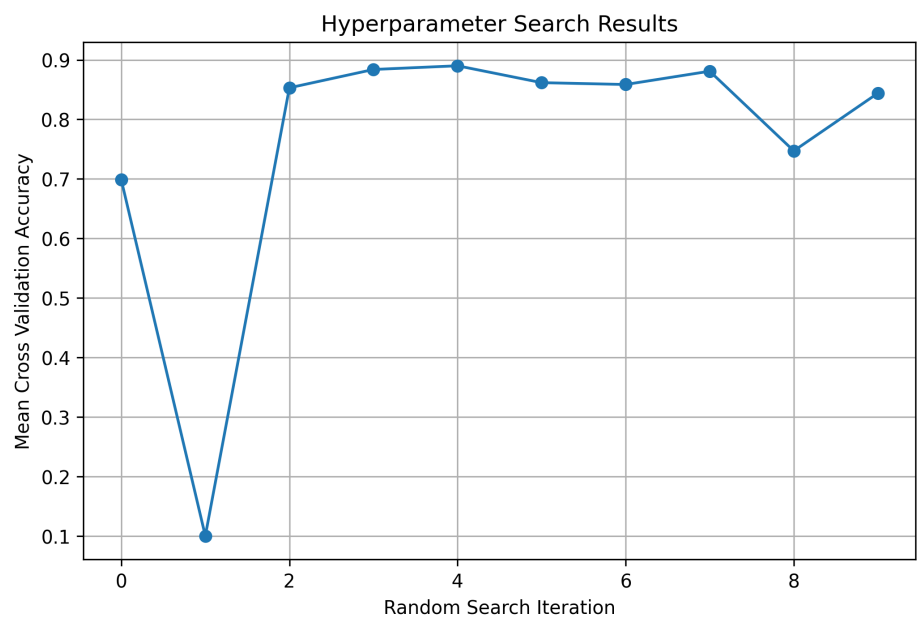


Figure 6: Hyperparameter Search Results

Inference: Different hyperparameter combinations gave different cross-validation accuracies. Some combinations performed poorly, while most of them gave accuracy close to 0.85–0.89. I selected the combination with the highest cross-validation accuracy as the best model for further testing.

7. Baseline vs Optimized Model Accuracy

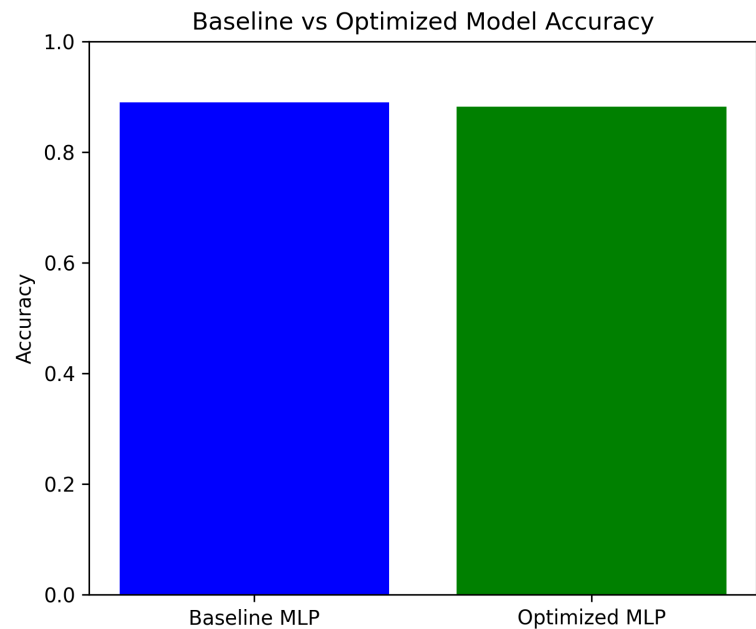


Figure 7: Baseline vs Optimized Model Accuracy

Inference: This graph compares the accuracy of the baseline model and the optimized model. In my experiment, both the optimized model and the baseline model achieved almost similar accuracy only, while the optimized model achieved a slightly lower testing accuracy than the baseline model, but the difference between them is very small..

9. Results

Best Hyperparameters

Hidden Layers	2
Hidden Neurons	128
Learning Rate	0.001
Batch Size	128
Optimizer	RMSProp
Activation Function	Sigmoid
Epochs	30
Dropout	0.2
Cross-validation Accuracy	0.8901
Testing Accuracy	0.8822

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8898	0.8822
Precision	0.8907	0.8832
Recall	0.8898	0.8822
F1-score	0.8878	0.8811
Training Time	Lower	Higher(Randomized Search)

10. Additional tasks:

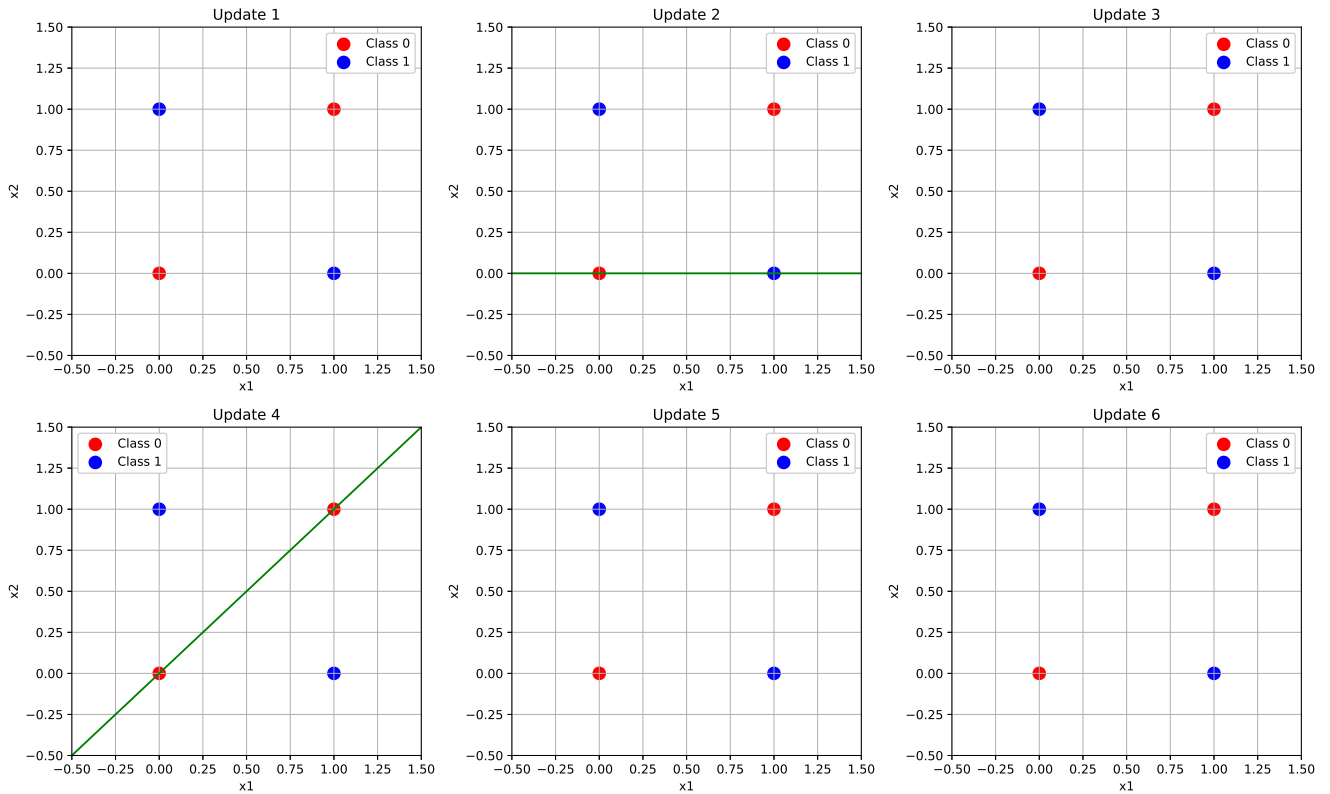
XOR Gate

Weight and Bias Updates

Update	Weights	Bias
1	[0.0, 0.0]	-0.1
2	[0.0, 0.1]	0.0
3	[-0.1, 0.0]	-0.1
4	[-0.1, 0.1]	0.0
5	[0.0, 0.1]	0.1
6	[-0.1, 0.0]	0.0
7	[-0.1, 0.0]	-0.1
8	[-0.1, 0.1]	0.0
9	[0.0, 0.1]	0.1
10	[-0.1, 0.0]	0.0
Final	[-0.1, 0.0]	0.0

The same sequence of weight and bias updates continued from Epoch 3 to Epoch 10 without reaching a stable solution.

XOR Gate - First 6 Updates



Inference:

Unlike the OR, AND and NOT gates, the XOR gate did not converge even after 10 epochs. The weights and bias kept repeating the same values instead of becoming stable. This happened because a Single Layer Perceptron can learn only one straight-line decision boundary, whereas the XOR data cannot be separated using a single straight line.

In the XOR gate, the inputs (0, 1) and (1, 0) belong to one class, while (0, 0) and (1, 1) belong to the other class. These points lie diagonally opposite to each other, so no single straight line can separate the two classes correctly. Because of this, every time the perceptron corrected one sample, another sample

became misclassified. Hence, the learning process continued repeating the same weight updates and the algorithm did not converge.

Therefore, the XOR problem cannot be solved using a Single Layer Perceptron. A Multilayer Perceptron (MLP) with one or more hidden layers is required to correctly classify the XOR gate.

11. Discussion

1. Which optimization method was used?

In this experiment, I used **Randomized Search** with 5-fold cross-validation for hyperparameter optimization. Instead of checking every possible combination, it selected few random combinations alone from given search space and checked. Each combination was tested using 5-fold cross-validation, and average accuracy was calculated. Finally, the combination with highest cross-validation accuracy was selected as the best set of hyperparameters. This method reduced the search time but still found a good model.

2. Which hyperparameters were selected?

The best hyperparameter combination selected in my experiment consists of 2 hidden layers, has 128 neurons and uses RMSProp optimizer, Sigmoid activation function, learning rate of 0.001, batch size of 128, 30 epochs and a dropout rate of 0.2. This combination achieved the highest cross-validation accuracy among all other combinations that were tested during Randomized Search. I then used this configuration to evaluate the optimized model on the testing dataset.

3. Did optimization improve performance?

Actually the optimized model achieved a testing accuracy of 88.22%, while the baseline model achieved 88.98%. The difference between the two models was very small, and the baseline model performed slightly better on the testing data. One possible reason is that the baseline model was already a good model for the Fashion-MNIST dataset. Although Randomized Search found the best combination based on cross-validation accuracy, it did not give a much better testing accuracy than the baseline model and were almost similar only.

4. Which hyperparameter had the greatest impact?

From my experiment, I observed that the optimizer, activation function and learning rate had the greatest effect on the model performance. Different combinations of these hyperparameters produced different cross-validation accuracies during Randomized Search. These parameters decide how the model learns from the training data, so changing them affected the final performance more than the other parameters.

5. Compare Grid Search and Randomized Search.

Grid Search checks every possible combination of hyperparameters, so it usually takes more time, mainly when many parameters are used. Randomized Search checks only limited number of randomly selected combinations, which makes it much faster. In this experiment, Randomized Search was a better choice because it reduced the computation time but also still found a good hyperparameter combination.

6. Recommend the best MLP configuration.

Randomized Search selected the configuration with 2 hidden layers, 128 neurons, RMSProp optimizer, Sigmoid activation function, learning rate of 0.001, batch size of 128, 30 epochs and a dropout rate of 0.2 because it achieved the highest cross-validation accuracy. However, when I compared both models on the testing dataset, I observed that the baseline model achieved a slightly higher testing accuracy. Therefore, both models performed well, but in this experiment the baseline model gave better final testing result.

12. Conclusion

In this experiment, a Multi-Layer Perceptron (MLP) model was implemented for classifying images from the Fashion-MNIST dataset. First, I loaded the dataset, displayed sample images and then performed preprocessing by flattening the images, normalizing the pixel values and converting the labels into one-hot vectors.

Next, I built a baseline MLP model and trained it using the Adam optimizer. After training, I evaluated the model using accuracy, precision, recall, F1-score and the confusion matrix. The baseline model achieved good performance and classified most of the images correctly.

Then hyperparameter optimization was performed using Randomized Search with 5-fold cross-validation. Different combinations of hyperparameters were tested and the best combination was selected based on the cross-validation accuracy. Although the optimized model performed well, its testing accuracy was slightly lower than the baseline model. This shows that hyperparameter tuning does not always improve the final testing accuracy by a higher margin, but always finds a good combination efficiently.

Overall, this experiment helped to understand how to preprocess image data, build and train an MLP model, evaluate its performance and use hyperparameter tuning to compare different model configurations.

13. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.